

Deliverable report

"Vittorio Polci": Mat: 257817, vittorio.polci@unitn.it, GitRepo: <https://github.com/vittorio01/GPU-Computing-2025-257817>

Abstract—This deliverable discusses about the implementation of different parallel solutions of the SpVM algorithm between CSR matrices and a general vector, with a final benchmark that compares all solution together.

Index Terms—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

I. INTRODUCTION

The SpMV (sparse matrix-dense vector multiplication) algorithm is an optimized form of matrix-vector multiplication tailored for matrices with a low density of non-zero entries. It is especially effective for very large sparse matrices, as it reduces storage requirements and improves memory access during computation. Given that such matrices often contain millions or billions of non-zero elements, achieving good performance requires parallelization on GPUs rather than relying on sequential CPU-based solutions.

II. PROBLEM STATEMENT

SpMV multiplication involves a matrix coded in a sparse format (in this case the CSR format) and an input vector matching the number of matrix columns. Each output element is obtained by computing the dot product between the non-zero entries of a matrix row and the corresponding elements in the input vector.

This report aims to present more CUDA-based solution for SpMV that matches different matrix distributions, along with advanced optimization techniques to enhance performance.

A. Storage Format

The CSR for sparse matrices is a storage format that involves three different arrays to represent the information:

- *The data array (dataArray)*, that contains all not-null terms of the original matrix.
- *The column array (colArray)*, where for each element in the data vector specifies its column coordinate.
- *The row array (rowArray)*, which contains the start and the end pointers for each row to the data and column arrays. these two pair of pointers indicates which not null terms in the data array are contained in that specific row.

The size of the data and column arrays depends on the number of not-null elements and the size of the row array depends on the number of the rows in the original matrix.

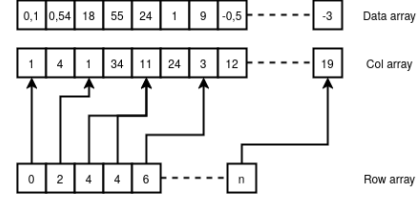


Fig. 1. Data organization in CSR structure

B. Parallelization

In the CUDA language model, used for programming on Nvidia [1] GPUs, enables running a kernel across many concurrent threads, which are managed and scheduled by CUDA cores, allowing efficient parallel execution.

To improve the performance and the organization of the parallel algorithms, CUDA architectures provides:

- *Warp organization*, where groups of 32 threads are able to synchronize and share data using predefined functions.
- *Block organization*, where more warps can synchronize themselves using atomic operations, barriers and have access to a dedicated shared memory.
- *Shared memory*, which is a fast but limited memory divided in more blocks and used to improve data reuse, optimize the global memory transfers and other purposes.
- *Read only cache*, which is a L1 read only cache that can be assigned by the programmer with the `__ldg()` function to improve data transfer for casual and repetitive accesses.

C. Warp reduction

In CUDA architectures there are predefined mechanisms to allow threads in the same warp to exchange data. The function `__shfl_down_sync()` permit to perform a reduction of arrays with maximum length of 32. For example, the function `reduceSum(acc)` uses five shuffle functions to accumulate all contribution given by threads in the same warp on the thread 0.

Algorithm 1 Warp reduction system

```
1: procedure REDUCESUM(acc)
2:   acc = __shfl_down_sync(0xffffffff, acc, 16)
3:   acc = __shfl_down_sync(0xffffffff, acc, 8)
4:   acc = __shfl_down_sync(0xffffffff, acc, 4)
5:   acc = __shfl_down_sync(0xffffffff, acc, 2)
6:   acc = __shfl_down_sync(0xffffffff, acc, 1)
7: end procedure
```

D. Ceiling division

The ceiling division is a system used to divide almost equally N elements across C equal size cells:

$$cellSize = \frac{N + C - 1}{C} \quad (1)$$

From the obtained division, the *start point* and the *end point* for each cell are defined by:

$$\begin{aligned} start &= cellSize \cdot cId \\ end &= \min(start + cellSize, N) \end{aligned} \quad (2)$$

E. Binary Search

The binary search is an algorithm that finds the position of a certain element in a sorted array with a complexity $O(\log(n))$. The proposed kernels use a modified version of the original algorithm to identify the row from a given index in the *dataArray* from the matrix.

Algorithm 2 Adapted binary search

```

1: procedure BINARYSEARCH(array, size, element)
2:    $index = -1, left = 0, right = size$ 
3:   while  $left \leq right$  do
4:      $mid = (left + right) / 2$ 
5:     if  $array[mid] \leq value$  then
6:        $index = mid$ 
7:        $left = mid + 1$ 
8:     else
9:        $right = mid - 1$ 
10:    end if
11:  end while
12: end procedure

```

III. METHODOLOGY AND CONTRIBUTIONS

A. Dynamic algorithm (Parallel-2)

In the first dynamic algorithm, the matrix and the vector are processed using a row based algorithm that assigns a thread for each row:

- 1) The *rowArray* is splitted in equal subsection for different blocks using the ceiling division (1).
- 2) For each block a variable in the shared memory *pendingRow* is initialized to 0 and used to contain the next row number that is waiting to be processed.
- 3) Each thread uses the function *atomicAdd(pendingRow, 1)* to increment the value of *pendingRow* and, at the same time, read its content.
- 4) Using the value obtained, each thread computes the products and the accumulation of the entire row, saves the result in the output array and repeats step 3 and 4 until the block reach the end of its subsection.

B. Optimized Dynamic algorithm (Parallel-5)

The dynamic algorithm can be improved further using a better organization of workload between threads. More performance can then be obtained by:

- Using the binary search in a preliminar kernel (6) to assign the rows to each block in base of the number of not-null elements of the matrix.

Algorithm 3 Dynamic SpVM algorithm

Require: input, output and matrix structures.

```

1: procedure VMCSR_MUL(output, matrix, input)
2:    $blockStart, blockEnd$  ▷ Ceiling division
3:    $pendingRow \leftarrow 0$  ▷ (only thread 0);
4:    $row = atomicAdd(pendingRow, 1) + blockStart$ 
5:   while  $row < blockEnd$  do
6:      $startRow, endRow$ 
7:      $\uparrow rowArray[row], rowArray[row + 1]$ 
8:      $acc = 0$ 
9:     while  $startRow < endRow$  do
10:       $col = colArray[startRow]$ 
11:       $acc +=$ 
12:         $dataArray[startRow] * input[col]$ 
13:       $startRow ++$ 
14:    end while
15:     $output[row] = acc$ 
16:     $row = atomicAdd(pendingRow, 1)$ 
17:     $+ blockStart$ 
18:  end while
19: end procedure

```

- Instead a single thread, Assigning a warp for each row. In this way, the threads in the warp process 32 partials from very long rows that can per reduced using the *reduceSum* function (1).
- Assigning the read only cache on the input vector to improve the speed by leveraging on the data reuse and using fused instructions.

Algorithm 4 Optimized dynamic algorithm (Parallel-5)

Require: matrix structure, block division array, input vector and output vector structures.

```

1: procedure VMCSR_MUL(output, matrix, input,
   blockDivision)
2:    $blockStart, blockEnd$  ▷ binary search kernel
3:    $pendingRow \leftarrow 0$  ▷ (only thread 0);
4:    $row = RowElection$ 
5:   while  $row < blockEnd$  do
6:      $startRow, endRow$ 
7:      $\uparrow rowArray[row] + laneId, rowArray[row + 1]$ 
8:      $acc = 0$ 
9:     while  $startRow < endRow$  do
10:       $col = colArray[idx]$ 
11:       $acc += fmaf($ 
12:         $dataArray[startRow], \_ldg(input[col]))$ 
13:       $startRow ++$ 
14:    end while
15:     $acc = ReduceSum$ 
16:    if  $laneId == 0$  then
17:       $output.dataArray[row] = acc$ 
18:    end if
19:     $row = RowElection$ 
20:  end while
21: end procedure

```

C. Shared cooperative algorithm (Parallel-6)

The dynamic assignment of the rows shows poor performances when considering heavily unbalanced matrices. Another approach that tries mitigate those cases divides the shared memory in three buffers, where the elements to

Algorithm 5 Row election System

```
1: procedure ROWELECTION
2:   if laneId == 0 then
3:     row = atomicAdd(pendingRow, 1)
4:     +blockStart
5:   end if
6:   __shfl_sync(0xffffffff, row, 0)
7: end procedure
```

Algorithm 6 Preliminar kernel

Require: matrix structure, block division array initialized on global memory.

```
1: procedure VMCSRDIV(matrix, blockDivision)
2:   blockStart, blockEnd      ▷ Ceiling on notNull array
3:   if blockIdx.x == 0 then
4:     blockDivision[0] = 0
5:     blockDivision[blockDim.x] = rowSize
6:   else
7:     blockDivision[blockIdx.x]
8:     ↑ binarySearch(rowArray, rowSize, blockStart)
9:   end if
10: end procedure
```

▷ Kernel launched for row division based on

binary search. The kernel should be launched with same number of block of the main kernel with size equal to 1 thread

load, store and compute are assigned equally to threads in the same block:

- *sharedBuffer*, a space of size $B = blockSize.x * N$ used as buffer to contain the result of the multiplication between elements of the matrix and the input vector (each thread operates at max on N elements).
- *loadedRows*, a space of $R = blockDim.x$ cells used as cache for parts of the CSR row vector (each threads loads one pointer).
- *rowsBuffer*, a space of $R - 1$ cells used for accumulating partials for a small amount of rows (almost each thread stores one cell in the global memory).

Using these three different shared spaces, each block performs these steps:

- 1) The entire block cooperates to *prepare the shared memory* for processing $R - 1$ rows by initializing with zeroes the *rowsBuffer* space and loading the necessary CSR pointers from the row array in the *loadedRows* space.
- 2) The threads *load a block of elements* (maximum B elements) from the matrix and the input vector in a fully coalesced way, perform the multiplications and store the result in *sharedBuffer*.
- 3) The loaded elements are first divided into equal strides using ceiling division (1). Each thread then performs a binary search (2) over the *loadedRows* array to find the row index corresponding to the first element of its stride. Subsequently, the strides are partially reduced and accumulated into the *rowsBuffer* using *atomicAdd*, based on the identified row indices.
- 4) The step 2 and 3 are performed until all necessary elements for the considered rows in *loadedRows* are

processed.

- 5) Each threads stores one result from the *rowsBuffer* to the output vector in the global memory.
- 6) All steps from 1 to 5 are repeated until all the rows of the matrix are processed.

The procedure 7 shows a heavily simplified representation of the shared cooperative algorithm.

Algorithm 7 Shared cooperative algorithm

Require: matrix structure, block division array, input vector and output vector structures.

```
1: procedure VMCSRMUL(output, matrix, input,
   blockDivision)
2:   blockStart, blockEnd      ▷ (From binary search kernel).
3:   sharedBuffer, rowsBuffer, loadedRows
4:   for [0, (R - 1)] rows in [blockStart, blockEnd] do
5:     if threadIdx.x < R then
6:       loadedRows[threadIdx.x] ← rowArray
7:       rowsBuffer[threadIdx.x] = 0
8:     end if
9:     for [0, B] in [loadedRows[0], loadedRows[R]] do
10:      for tId in [0, B], coalesced do
11:        sharedBuffer[tId] ← dataArray[tId]
12:        * __ldg(input[colArray[tId]])
13:      end for
14:      tStart, tEnd ← Ceiling on B
15:      row ← binarySearch(loadedRows, B, tStart)
16:      for i in [tStart, tEnd] do
17:        row ← row adjusting
18:        atomicAdd(rowsBuffer[row], i)
19:      end for
20:    end for
21:    if threadIdx.x < (R - 1) then
22:      output ← rowsBuffer[threadIdx.x]
23:    end if
24:  end for
25: end procedure
```

IV. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

A. Dataset description

A dataset composed by 6 matrices, obtained from Matrix Collection [2], is used for testing and benchmark all four algorithms. These matrices contains a number of not-null elements between 1 and 1,5 million and they have different organization and dispersion that challenges the algorithms in different ways.

Dataset	Dimension (R x C)	Non-zero entries	Type
dbir2 [3]	18,906 x 45,887	1,158,159	Unbalanced rows
ex11 [4]	16,614 x 16,614	1,096,948	Diagonally distributed (low density)
language [5]	399,130 x 199,130	1,216,334	Upper triangular
Linux_call_graph [6]	324,085 x 324,085	1,208,908	Dense matrix (worst case)
nemeth24 [7]	9,506 x 9,506	1,506,550	Diagonally distributed (best case)
twotone [8]	120,750 x 120,750	1,206,265	Diagonally distributed (moderate dispersion)

TABLE I
STRUCTURAL PATTERNS OF THE BENCHMARK SPARSE MATRICES.

B. System Description and Experimental Set-up

The system used for the execution of the benchmarks uses CUDA 12.3.2 for compiling the source code, and is composed

by a Intel(R) Xeon(R) Silver 4214 CPU and an Nvidia L40S GPU. The Nvidia L40S is based on Ada Lovelace architecture and has 142 Stream Multiprocessors with 96KB available shared memory per block, maximum memory bandwidth of 864GB/s and 91,61TFlops for FP32 operations.

The performance is measured by considering the GFlops obtained by the sum of the execution times of all kernels (preliminar and main kernels) measured with *cudaEventRecord* functions for CUDA algorithms and considering also the DRAM throughput and Compute utilization obtained from Nvidia Nsight Compute [9]. The results obtained by the three kernels are then compared with the library cuSPARSE [10], the official library supported by Nvidia [1].

C. Block size estimation

The parameters used for launching the kernels are calculated empirically in base of the amount of shared memory available per block, the number of theoretical threads that should run simultaneously in the same SM and the size of the input matrix. In particular:

- The number of blocks to be launched is equal to the number of not-null elements of the input matrix divided by a constant obtained empirically of around 3328 (*notNull/NotNullPerBlock*).
- In the kernel 7, the dimension of the shared memory is related on the number of elements that each thread should load and the block dimension ($shared = (elementsPerThread + 2) * blockDim * sizeof(float)$).
- We can obtain the number of threads per block by considering N block active at the same time in a Stream Multiprocessor, which is limited to the shared memory for the kernel *parallel-6*. In this case we consider the number of threads equal to $sharedPerSM(Kb) \div [N * (elementsPerThread + 2) * sizeof(float)]$. From various benchmarks, the best performances are obtained when considering $N = 2$ and $elementsPerThread = 14$. For the kernels *parallel-2* and *parallel-5* the number of threads can also be fixed at a value of 512 or 256 to obtain stable performances.

V. EXPERIMENTAL BENCHMARKS

The benchmarks obtained by launching the four algorithms are represented in the three tables in figure 2.

VI. RESULTS AND CONCLUSION

The benchmarks shows that:

- The algorithm *parallel-2* performs better when applied to balanced diagonal matrices with short rows, such as *ex11* [4], *twotone* [8], or *nemeth24* [7]. This is because a single thread can efficiently handle the computational tasks associated with small rows. Moreover, since smaller rows are more likely to be concentrated within a single DRAM bank, more threads can operate concurrently during a

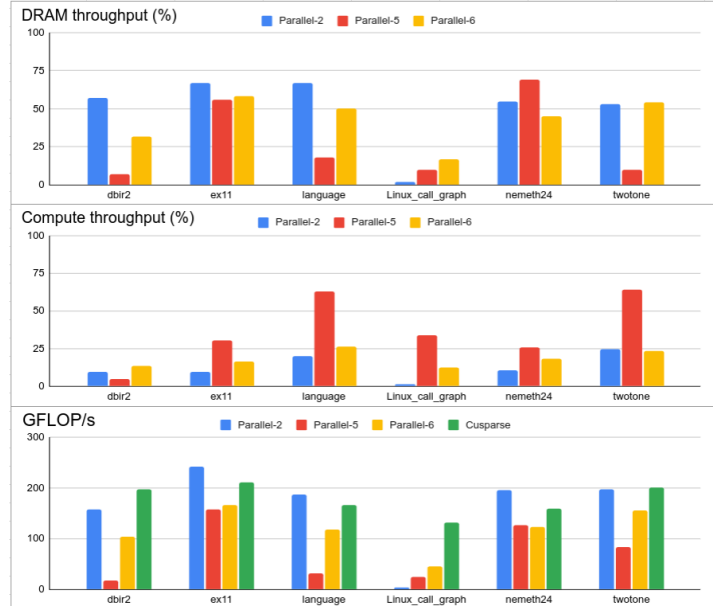


Fig. 2. Performance differences between the four algorithms

single memory transfer. On the other hand, the performance on unbalanced matrices like *Linux_call_graph* [6] are slightly decreased, especially because the uncoalesced requests from the DRAM (figure 2 shows a decrease of the DRAM throughput).

- The algorithm *parallel-5* shows that the performance for *Linux_call_graph* [6] thanks to its warp centric structure. Now, because 32 threads operates on the same rows, the workload is more balanced and the memory access are more concentrated (condition that improve the DRAM throughput and the Compute throughput). But, because entire warps are assigned per row, well balanced matrices like *dbir2* [3] or *twotone* [8] shows worse performance if compared with *parallel-2*.
- The *parallel-6* algorithm employs a different strategy, using perfectly coalesced memory accesses and workload distribution via a shared buffer. This leads to significantly better performance on heavily unbalanced matrices like *Linux_call_graph* [6]. However, the kernel is more latency-bound due to atomic operations causing contention when many threads accumulate partial results in the same row, as reflected by similar DRAM and Compute throughput values.

In conclusion, the *parallel-2* algorithm delivers faster performance on regular and diagonal matrices, while *parallel-6* demonstrates adaptability across the tested datasets. Revisiting the workload distribution among threads (especially the accumulation process in rowBuffer implemented in the kernel 7) and implementing an automatic algorithm selection mechanism, the general performance levels could achieve results close to those of cuSPARSE [10].

REFERENCES

- [1] Nvidia. [Online]. Available: <https://www.nvidia.com/it-it/>
- [2] U. of Florida. Suitesparse matrix collection. [Online]. Available: <https://sparse.tamu.edu/>
- [3] Meszaros. dbir2 matrix. [Online]. Available: <https://sparse.tamu.edu/Meszaros/dbir2>
- [4] FIDAP. ex11 matrix. [Online]. Available: <https://sparse.tamu.edu/FIDAP/ex11>
- [5] Tromble. language matrix. [Online]. Available: <https://sparse.tamu.edu/Tromble/language>
- [6] Sorensen. Linux call graph matrix. [Online]. Available: https://sparse.tamu.edu/Sorensen/Linux_call_graph
- [7] Nemeth. nemet24 matrix. [Online]. Available: <https://sparse.tamu.edu/Nemeth/nemeth24>
- [8] ATandT. twotone matrix. [Online]. Available: <https://sparse.tamu.edu/ATandT/twotone>
- [9] Nvidia nsight compute. [Online]. Available: <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>
- [10] NVIDIA. cusparse. [Online]. Available: <https://developer.nvidia.com/cusparse>